# 🧷 Discard Your Changes

*(Descartar mudanças no código)*

Aqui você aprende a:

❗ apagar alterações — de forma controlada

Se usado errado → você perde trabalho

Se usado certo → você salva tempo e evita bagunça

# 🧠 Ideia central

Existem 3 tipos de coisas que você pode querer descartar:

1. alterações em arquivos (não commitadas)
2. alterações já adicionadas (`git add`)
3. tudo (reset total)

# 💧 1. `git restore` — apagar mudanças em arquivos

### 📌 Comando:

```
git restore arquivo.txt
```

### 🧠 O que faz?

👉 Volta o arquivo para o último estado salvo

### 🔧 Exemplo

Antes:

```
preço = 10
preço = 20  ← você mudou
```

Depois:

```
git restore app.txt
```

Resultado:

```
preço = 10
```

### ⚠️ Atenção

Isso apaga suas alterações locais

## 🧬 2. `git restore --staged` — tirar do staging

### 📌 Comando:

```
git restore --staged arquivo.txt
```

## 🧠 O que faz?

👉 Remove do `git add`, mas mantém o código

## 🔧 Exemplo

```
git add app.txt
git restore --staged app.txt
```

👉 arquivo sai da staging, mas continua editado

# 💣 3. `git restore --staged --worktree`

### 📌 Comando:

```
git restore --staged --worktree arquivo.txt
```

## 🧠 O que faz?

👉 Apaga TUDO do arquivo:

- staging
- alterações

## 💡 Tradução

“volta completamente ao estado do último commit”

# 🧪 4. `git reset --hard` — destruição total

### 📌 Comando:

```
git reset --hard
```

## 🧠 O que faz?

👉 Apaga:

- alterações
- staging
- volta tudo ao último commit

## ⚠️ PERIGO

Isso pode apagar horas de trabalho

## 💧 Quando usar?

- você bagunçou tudo
- quer voltar ao estado limpo

# 🧹 5. `git clean` — apagar arquivos não rastreados

### 📌 Comando:

```
git clean -f
```

## 🧠 O que faz?

👉 remove arquivos que o Git não controla

## 🔧 Exemplo

Arquivos:

```
teste.txt
log.txt
git clean -f
```

👉 deletados

## ⚠️ Atenção

Use com cuidado — ele apaga arquivos do disco

## 🧠 6. Comparação clara

| Situação | Comando |
|---|---|
| desfazer edição | `git restore arquivo` |
| tirar do add | `git restore --staged` |
| apagar tudo do arquivo | `git restore --staged --worktree` |
| reset total | `git reset --hard` |
| apagar arquivos extras | `git clean -f` |

## 💧 Exemplo completo (vida real)

```
echo "teste" >> app.txt

git status
```

Você errou:

```
git restore app.txt
```

Outro caso:

```
git add .
git restore --staged .
```

## 🧠 Cenário real (nível profissional)

Você está trabalhando e:

- testou algo errado
- quer desfazer rápido

## Estratégia:

👉 pequeno erro:

```
git restore arquivo.txt
```

👉 erro médio:

```
git reset
```

👉 caos total:

```
git reset --hard
```

# ⚠️ Erros comuns

### ❌ usar `reset --hard` sem pensar

→ perde tudo

### ❌ usar `clean` sem saber

→ apaga arquivos importantes

### ❌ não revisar antes

→ perde código útil

# 🧭 Regra de ouro

Nunca use comandos destrutivos sem saber o que vai perder

# 🧠 Nível engenharia

Agora você sabe:

- apagar mudanças com controle
- limpar ambiente
- recuperar organização do projeto

👉 Isso é essencial para trabalhar em projetos reais